

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO4: Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)

Assessment Tool Weightage: 20%

CO4-AT(2)

QUESTIONS: 5
Duration : 1 Hour

Total Marks:25

Industry Problem

Code of Conduct:

I Mysai Reddy T / 192372252 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

SIMATS ENGINEERING ASSESSMENT TOOLS

1. Design and implement a **C-based validation module** that simulates a **Deterministic Finite Automaton (DFA)** to verify input strings in a real-time system. The module should accept only those strings that **begin with the character 'a' and end with the character 'a'**, ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- Process user input dynamically
- Validate strings based on DFA state transitions
- Output whether the given input is **Accepted** or **Rejected**

2. Design a Push Down Automata that accepts the language

$$L = \{w \mid w \in (0 + 1)^* \text{ and } n_0(w) = n_1(w)\}$$

$n_0(w)$ is the number of 0's in w

$n_1(w)$ is the number of 1's in w

3. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice than that of a's.

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

4. Develop a **C-based computational module** to determine the **ϵ -closure of all states** in a **Non-Deterministic Finite Automaton (NFA) with ϵ -transitions**, as part of an automata processing engine used in compiler construction and pattern-matching systems.

5. Develop a **C-based input validation module** that determines whether a given binary string conforms to a specified **Context-Free Grammar (CFG)** used in structured data validation systems.

The grammar is defined as:

- $S \rightarrow 0 A 1$
- $A \rightarrow 0 A \mid 1 A \mid \epsilon$

DFA for strings and Ending with a language:-

$$L = \{w \in \{a, b\}^* \mid w \text{ states with and ends with } a\}$$

DFA states:-

= * = * = *

$\Rightarrow q_0$ - I.S

$\Rightarrow q_1$ - Starting with a and currently ends with a (final state).

$\Rightarrow q_2$ - String start with a but currently ends with b.

$\Rightarrow q_d$ - Dead state.

Transition table:-

= * = * = * = *

State	a	b
$\rightarrow q_0$	q_1	q_d
$* q_1$	q_1	q_2
q_2	q_2	q_2
q_d	q_d	q_d

Program:-

= * = * =

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main () {
```

```
char str [100];
```

```

int main () {
    char str[100];
    int state = 0;
    printf ("Enter a string: ");
    scanf ("%s", str);
    switch (state) {
        case 0 :
            if (ch == '0')
                state = 1;
            else
                state = 3;
            break;
        case 1 :
            printf ("Accepted\n");
            return 0;
        case 3 :
            printf ("Rejected\n");
            return 0;
    }
}

```

②

Pushdown Automaton for Equal No. of 0's and 1's.

$$L = \{w \in \{0,1\}^+ \mid n_0(w) = n_1(w)\}$$

where

$$\Rightarrow n_0(w) = \text{no. of 0's in } w.$$

$$\Rightarrow n_1(w) = \text{no. of 1's in } w.$$

PDA Idea:-

= * = * = *

The PDA the stack of to the cancel opposite Symbols

When 0 is read, push 0 if the top is Zero otherwise Pop().

is read, Push 1 if the top is Zero (or) 1;
otherwise Pop 0.

Normal PDA:-
* = * = *

$$M = (\{q\}, \{0,1\}, \{0,1,z_0\}, \delta, q, z_0, F)$$

The PDA accepts when the stack returns to the initial stack symbol z_0 , No. of 0's & 1's is Equal.

Ex:-

- $\Rightarrow 01 \rightarrow$ Accepted.
- $\Rightarrow 10 \rightarrow$ Accepted.
- $\Rightarrow 0011 \rightarrow$ Accepted.
- $\Rightarrow 0101 \rightarrow$ Accepted.
- $\Rightarrow 001 \rightarrow$ ~~Rejected.~~

③

PDA for $L = a^n b^{2n} \mid n \geq 1$

Language:-

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

For every a , there must be exactly two b 's.

PDA strategy:-
* = * = *

- ① For every a , push two stack symbols x .
- ② For every b , pop one x .
- ③ Accept when all input is processed and the stack reaches the bottom symbol.

Transitions:-

$= * \leftarrow * = * =$

For Each a:

$$\delta(q_0, a, z_0) = \{q_0, xxz_0\}$$

$$\delta(q_0, a, x) = (q_0, xxx)$$

When b starts:

$$\delta(q_1, b, x) = (q_1, \epsilon)$$

for remain b's:

$$\delta(q_1, b, x) = (q_1, \epsilon)$$

Accept when stack reaches z_0 :

$$\delta(q_1, \epsilon, z_0) = (q_f, z_0)$$

Ex:- For the string aabbx Rejected because:

$2a \Rightarrow 4b$ are required.

For string aabbbb; $2a \Rightarrow 4b$

$\therefore$ aabbbb $\rightarrow$ Accepted.

(4)

Program to find ϵ -closure of All states in NFA.

```
#include <stdio.h>
```

```
#define MAX 20
```

```
int n;
```

```
int E[MAX][MAX];
```

```
int visited[MAX];
```

```
void  $\epsilon$ -closure (int state) {
```

```
    int i;
```

```
    visited[state] = 1;
```

(3)

```
for (i=0; i<n; i++) {
```

```
    if (Epsilon[state][i] == 1 && ! visited[i]) {
```

```
        epsilon_closure[i];
```

```
    }
```

```
 }
```

```
 }
```

```
int main () {
```

```
    int i, j, state;
```

```
    printf ("Enter no. of states: ");
```

```
    scanf ("%d", &n);
```

```
    for (i=0; i<n; i++) {
```

```
        for (j=0; j<n; j++) {
```

```
            scanf ("%d", &Epsilon[i][j]);
```

```
        }
```

```
    printf ("In epsilon closure of i %d = ", state);
```

```
    epsilon_closure(state);
```

```
    printf (" ");
```

```
 }
```

```
return 0;
```

```
 }
```

Qf:-

$q_0 \rightarrow q_1 \rightarrow q_2$

Through ϵ -transition then:

ϵ -closure(q_0) = $\{q_0, q_1, q_2\}$;

C-Program :-

=* =* =*

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main () {
```

```
    char str [100];
```

```
    int len, valid = 1, i;
```

```
    printf ("Enter a binary string: ");
```

```
    scanf ("%s", str);
```

```
    len = strlen (str);
```

```
    if (len < 2) {
```

```
        valid = 0;
```

```
    }
```

```
    if (str [len-1] != '1') {
```

```
        valid = 0;
```

```
    }
```

```
    for (i = 0; i < len; i++) {
```

```
        valid = 0;
```

```
        break;
```

```
    }
```

```
    printf ("String is Accepted \n");
```

```
    else
```

```
    printf ("String is Rejected \n");
```

```
    return 0;
```

```
}
```

Examples :-

=* =* =*

I/P =* =*	Result =* =*
01	Accepted
001	Accepted
011	Accepted
010101	Accepted
10	Rejected
10	Rejected

SIMATS ENGINEERING ASSESSMENT TOOLS

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

92/100

JH